React Native and TypeScript Practice Workbook

Coding exercises, hints, model answers, and mock interviews

Independently prepared study materials based on the ten supplied Autumn 2026 lecture decks. Sources checked 6 November 2026. The practice questions and rubrics are original and are not official assessments.

Work through a prompt before reading its hints or reference answer. Each solution is labeled as a complete file or a fragment. Run code in your existing course app, test the stated behavior, and explain why the solution works. Adapt patterns to new requirements rather than memorizing an answer.

The companion HTML study hub helps you choose a skill, trace an example, and move into practice. It includes editable scratchpads, search, collapsible answers, and an optional 25-minute timer. Exercise durations below are estimates, not deadlines.

For the five core skills, choose E01 (arrow functions), E02 (map), E04 (array updates), E07 (callback props), or E08 (inference). Combine them in E10 or a mock interview when useful. You can move between lessons and exercises freely; there is no required order.

Exercise index

E01 Translate arrow functions without guessing / 8 minutes

E02 Repair a map that displays nothing / 10 minutes

E03 Make a counter update through React / 10 minutes

E04 Append without mutating the original array / 10 minutes

E05 Update one object and remove another / 12 minutes

E06 Connect a controlled input / 10 minutes

E07 Send an action from child to parent / 14 minutes

E08 Place types where inference needs help / 10 minutes

E09 Build a nested layout from a verbal description / 15 minutes

E10 Rebuild the complete campus menu / 25 minutes

E11 Fix imports and component props / 8 minutes

E12 Debug rendering and event mistakes / 10 minutes

E13 Run one interval and clean it up / 15 minutes

E14 Predict and chain Promises / 12 minutes

E15 Insert and replace with slice / 10 minutes

E16 Explain a cross-platform implementation choice / 8 minutes

E01 Translate arrow functions without guessing

Essential / 8 minutes / arrow functions, return, objects

This is a standalone TypeScript drill. Write each function in both expression-body and block-body form. First predict the results of triple(4), label("Tea"), and makeItem("Tea"). Then explain every pair of parentheses and braces out loud.

- Return 12 from triple(4).
- Return "Item: Tea" from label("Tea").
- Return an object with name "Tea" from makeItem("Tea").
- Use no any types.

Starter code

```
const triple = (n: number) => { n * 3; };
const label = (name: string) => { return "Item: " + name; };
const makeItem = (name: string) => { name: name };
```

Plan before reading the answer

Identify the input, the output, the state owner, and the event that changes the state. Write your solution in your editor before continuing.

Hint 1: A block body needs an explicit return when a result is required.

Hint 2: An object expression begins with a brace, so wrap the object in parentheses in an expression body.

Hint 3: Compare n => n * 3 with n => { return n * 3; }.

Reference answer

```
// Standalone TypeScript.
const triple = (n: number) => n * 3;
const tripleBlock = (n: number) => { return n * 3; };
const label = (name: string) => "Item: " + name;
const labelBlock = (name: string) => { return "Item: " + name; };
const makeItem = (name: string) => ({ name });
const makeItemBlock = (name: string) => { return { name }; };
console.log(triple(4), label("Tea"), makeItem("Tea"));
```

The body after => is either one expression or a statement block. An expression body returns its value automatically. A block executes statements and needs return to provide a value. Without return, triple originally produced undefined.

In the original makeItem, the braces start a block and name: is parsed as a label, not an object property. Parentheses make ({ name }) an object expression. The shorter property name form means { name: name }.

Test the behavior

- The three printed values are 12, "Item: Tea", and { name: "Tea" }.
- Change the input to 0 and the empty string; the functions still return values.

- Rewrite all three without looking, then compare expression and block results.

Transfer challenge

Write a function that returns { title, done: false } for a study task. Explain why => ({ ... }) works.

E02 Repair a map that displays nothing

Essential / 10 minutes / map, JSX, keys

Repair this JSX fragment inside a component. Assume Text and View are imported from react-native. Explain what each invocation of the map callback receives and returns. Write both valid callback styles.

- Display Tea and Soup in that order.
- Use a stable key from the data.
- Do not alter items or use a for loop.

Starter code

```
const items = [{ id: "a", name: "Tea" }, { id: "b", name: "Soup" }];
return (
  <View>
    {items.map(item => { <Text>{item.name}</Text>; })}
  </View>
);
```

Plan before reading the answer

Identify the input, the output, the state owner, and the event that changes the state. Write your solution in your editor before continuing.

Hint 1: Inspect the callback return value, not only the JSX element.

Hint 2: JSX braces insert an expression into the parent View. They are different from a function-body block.

Hint 3: Use item => (<Text key={item.id}>{item.name}</Text>) or a block with return.

Reference answer

```
// Component-body fragment; Text and View are imported.
const items = [{ id: "a", name: "Tea" }, { id: "b", name: "Soup" }];
const rows = items.map(item => (
  <Text key={item.id}>{item.name}</Text>
));
return <View>{rows}</View>;

// Equivalent map expression, used instead of the one above:
// items.map(item => {
//   return <Text key={item.id}>{item.name}</Text>;
// });
```

map calls the callback for each present array element and gathers the returned values into a new array. Here the input elements are data objects; the returned values are React elements. The original callback returned undefined, so it supplied no text elements to display.

The outer {rows} inserts the array into JSX. key helps React match siblings across changes. It belongs on the outermost element returned by this map, and a data ID is more stable than a changing position or a random number.

Test the behavior

- Two text rows appear, with no missing-key warning.
- Reverse the input array; the rendered order reverses.
- Temporarily remove return from a block callback and explain the resulting empty display.

Transfer challenge

Render a list of campus clubs using a custom ClubCard component instead of Text.

E03 Make a counter update through React

Essential / 10 minutes / useState, events, state snapshots

Build a complete App.tsx counter. The count starts at zero, Add increments once, Add three increments three times, and Reset sets zero. Explain why changing a local variable is insufficient.

- Use state at the top level of the component.
- Pass functions to onPress.
- Use three queued functional updates for Add three.

Starter code

```
// Broken component-body fragment:
let count = 0;
const add = () => { count += 1; };
```

Plan before reading the answer

Identify the input, the output, the state owner, and the event that changes the state. Write your solution in your editor before continuing.

Hint 1: A local variable is recreated when the component runs, and assigning it does not schedule a render.

Hint 2: useState returns the current render value and a setter.

Hint 3: Three calls to setCount(c => c + 1) compose; three calls to setCount(count + 1) use the same captured value.

Reference answer

```
// Complete App.tsx.
import { useState } from 'react';
import { Button, Text, View } from 'react-native';
export default function App() {
  const [count, setCount] = useState(0);
  function addThree() {
    setCount(c => c + 1);
    setCount(c => c + 1);
    setCount(c => c + 1);
  }
  return (
    <View style={{ padding: 40 }}>
      <Text>{count}</Text>
      <Button title="Add" onPress={() => setCount(c => c + 1)} />
      <Button title="Add three" onPress={addThree} />
      <Button title="Reset" onPress={() => setCount(0)} />
    </View>
  );
}
```

React retains state between renders. Calling its setter queues an update; the next render receives the next value. The count variable in the current handler remains the current render's snapshot.

A setter accepts either the next value or an updater function. Each updater receives the pending state after earlier queued updates. This is why three functional increments produce three additional counts. An updater should be pure: return the next value without changing external data.

Test the behavior

- Start at 0; Add gives 1; Add three gives 4; Reset gives 0.
- Replace the three updater calls with setCount(count + 1); from 0 they produce 1. Explain before running.
- A log of count immediately after calling its setter still shows the current handler's captured value.

Transfer challenge

Build a ticket counter with a decrement button that never goes below zero.

E04 Append without mutating the original array

Essential / 10 minutes / array spread, functional updates, inference

Repair this attempted update. This is a component-body fragment with useState imported. Show both a direct replacement and a functional update. Prefer the latter for the final add handler because the next menu depends on the previous menu.

- Append to an array, not an object.
- Preserve the old array and its entries.
- Explain why TypeScript infers MenuItem[] here.

Starter code

```
type MenuItem = { name: string; popular: boolean };
const initialMenu: MenuItem[] = [{ name: "Tea", popular: false }];
const [menu, setMenu] = useState(initialMenu);
const addMenuItem = (item: MenuItem) => {
  setMenu(previous => { ...previous, item });
};
```

Plan before reading the answer

Identify the input, the output, the state owner, and the event that changes the state. Write your solution in your editor before continuing.

Hint 1: The setter can accept an updater. Keep that idea and fix its returned value.

Hint 2: Square brackets construct an array. Spread copies its elements into a new array.

Hint 3: Return [...previous, item] with an expression body, or use a block with return.

Reference answer

```
// Component-body fragment; useState is imported.
type MenuItem = { name: string; popular: boolean };
const initialMenu: MenuItem[] = [{ name: "Tea", popular: false }];
const [menu, setMenu] = useState(initialMenu);
const addMenuItem = (item: MenuItem) => {
  setMenu(previous => [...previous, item]);
};
// Also valid for one update based on this render's menu:
// setMenu([...menu, item]);
// Equivalent functional form:
// setMenu(previous => { return [...previous, item]; });
```

Both a direct array replacement and a functional update are valid. Lecture 5a slide 10 explicitly teaches both forms. The starter fails because it uses a malformed block/spread expression instead of returning an array. Square brackets construct the array; an expression body returns it automatically.

Spreading an array into another array copies element references; it does not deep-copy nested objects. That is enough for appending a new item, provided old objects are not mutated. The typed initialMenu provides information from which useState infers MenuItem[].

Test the behavior

- Add Soup after Tea; the next array contains both in order.
- Keep a reference to the old array in a standalone equivalent; its length remains 1 and the next array is a different reference.
- Queue two functional appends in the same event; both items remain.

Transfer challenge

Prepend the new item, then explain the difference between [item, ...previous] and [...previous, item].

E05 Update one object and remove another

Essential / 12 minutes / map, filter, object spread, immutability

Write two handlers inside a component that stores Task[] in state. toggleTask(id) changes only the matching task's done value. removeTask(id) deletes only that task. The IDs are unique.

- Return new arrays.
- Copy the changed object instead of mutating it.
- Keep IDs stable.
- Do nothing harmful when an ID is absent.

Starter code

```
type Task = { id: string; title: string; done: boolean };
const [tasks, setTasks] = useState<Task[]>([
  { id: "t1", title: "Read", done: false },
  { id: "t2", title: "Practice", done: true },
]);
// Write toggleTask and removeTask here.
```

Plan before reading the answer

Identify the input, the output, the state owner, and the event that changes the state. Write your solution in your editor before continuing.

Hint 1: map preserves a one-for-one relationship; filter selects which elements remain.

Hint 2: For the matching task return { ...task, done: !task.done }.

Hint 3: The filter callback should return true for tasks to keep.

Reference answer

```
// Component-body fragment; useState is imported and tasks exist.
function toggleTask(id: string) {
  setTasks(previous => previous.map(task =>
    task.id === id ? { ...task, done: !task.done } : task
  ));
}
function removeTask(id: string) {
  setTasks(previous => previous.filter(task => task.id !== id));
}
```

An immutable edit copies each level that changes: the array and the matching task object. Unchanged task objects may retain their references because their values are unchanged. [...previous] followed by next[0].done = true would still mutate the shared first object.

filter keeps elements whose predicate is true. To remove t1, keep tasks with task.id !== "t1". map transforms elements; it does not remove them just because a callback returns undefined.

Test the behavior

- Toggle t1: t1 becomes true and t2 remains true. Toggle t1 again: it returns to false.
- Remove t2: only t1 remains. Removing an unknown ID leaves the displayed data unchanged.
- Save the old t1 object in a standalone test and confirm its done field did not change.

Transfer challenge

Insert a task at index 1 with slice, then replace an item at that index without using map.

E06 Connect a controlled input

Essential / 10 minutes / TextInput, controlled state, callbacks

Create a complete screen that displays a name as it is typed and clears it with a button. The button is disabled when the input is empty. Explain why onChangeText receives a string rather than a browser event.

- Keep value and onChangeText connected to the same state.
- Render text inside Text.
- Do not call the setter while rendering.

Starter code

```
// In a component:
// <TextInput value={name} onChangeText={/* ? */} />
```

Plan before reading the answer

Identify the input, the output, the state owner, and the event that changes the state. Write your solution in your editor before continuing.

Hint 1: Initialize name with useState("").

Hint 2: Pass setName directly because its accepted string value fits this callback.

Hint 3: For Clear, pass () => setName("") rather than setName("").

Reference answer

```
// Complete App.tsx.
import { useState } from 'react';
import { Button, Text, TextInput, View } from 'react-native';
export default function App() {
  const [name, setName] = useState('');
  return (
    <View style={{ padding: 40 }}>
      <TextInput accessibilityLabel="Your name" value={name}
        onChangeText={setName}
        style={{ borderWidth: 1, padding: 12 }} />
      <Text>Hello, {name || 'friend'}.</Text>
      <Button title="Clear" disabled={name.length === 0}
        onPress={() => setName('')} />
    </View>
  );
}
```

The displayed input value comes from React state. Typing invokes onChangeText with the new string; setName schedules a render; the next value prop reflects that string. This complete loop makes the input controlled.

React Native TextInput has onChangeText for direct text values. A browser input often uses an event and event.target.value; copying that web pattern into onChangeText would be incorrect. A value prop without a corresponding update can make the input appear stuck.

Test the behavior

- Typing Alex changes both the input and greeting.
- Clear empties the field, changes the greeting to friend, and disables itself.
- Typing a space is allowed by this exercise; explain how trim() would change validation.

Transfer challenge

Add a character counter derived from name.length without introducing another state variable.

E07 Send an action from child to parent

Essential / 14 minutes / callback props, lifting state, function types

Complete a parent and child pair. The parent owns an array of names. The child owns only the draft text. Pressing Add in the child should ask the parent to append the trimmed name. The array must remain in the parent.

- Type onAdd as a function that receives one string and returns no useful value.
- Reject blank or whitespace-only submissions.
- Clear the child draft after a successful submission.
- Use a functional array update in the parent.

Starter code

```
type ComposerProps = { onAdd: /* write the function type */ };
function Composer({ onAdd }: ComposerProps) {
  // Own draft text; call onAdd only on submission.
}
export default function App() {
  // Own names and pass an add handler to Composer.
}
```

Plan before reading the answer

Identify the input, the output, the state owner, and the event that changes the state. Write your solution in your editor before continuing.

Hint 1: The type is (name: string) => void. That describes the callback; it does not execute it.

Hint 2: The parent passes onAdd={addName}. The child later calls onAdd(cleanName) in a handler.

Hint 3: Props travel down; an event invokes a function supplied from above. The child does not directly write the parent's state.

Reference answer

```
// Complete App.tsx. The names below are append-only; a single Text displays
them.
import { useState } from 'react';
import { Button, Text, TextInput, View } from 'react-native';
type ComposerProps = { onAdd: (name: string) => void };
function Composer({ onAdd }: ComposerProps) {
  const [draft, setDraft] = useState('');
  function submit() {
    const name = draft.trim();
    if (!name) return;
    onAdd(name);
    setDraft('');
  }
  return (
    <View>
      <TextInput accessibilityLabel="New name" value={draft}
        onChangeText={setDraft} style={{ borderWidth: 1, padding: 12 }} />
      <Button title="Add" onPress={submit} disabled={!draft.trim()} />
    </View>
  );
}
export default function App() {
  const [names, setNames] = useState<string[]>([]);
  function addName(name: string) {
    setNames(previous => [...previous, name]);
  }
  return (
    <View style={{ padding: 40 }}>
      <Composer onAdd={addName} />
      <Text>{names.length ? names.join(', ') : 'No names yet'}</Text>
    </View>
  );
}
```

There are two pieces of state with different owners. The draft is used only inside the form, so it stays there. The submitted names are needed by the parent display, so they live in App. If two siblings later need the same draft, move it to their nearest common ancestor.

onAdd is a prop containing a function value. Calling it reports an action to the parent's handler. This does not reverse the direction of props: the parent still supplies both data and callbacks downward, and owns the state update.

Test the behavior

- Type two names and submit each. Both appear in order.
- Submitting spaces adds nothing.
- Explain the path from the press, through submit and onAdd, to setNames and the next render.

Transfer challenge

Change the callback to accept { name, popular } instead of a string and adjust both its type and the object passed by the child.

E08 Place types where inference needs help

Essential / 10 minutes / TypeScript, inference, union types, props

Complete this standalone TypeScript and React-state typing drill. Explain which annotations are needed for the intended future values, and which can be inferred. Do not add any or assertions to silence errors.

- Allow items to hold MenuItem objects.
- Allow selected to hold a MenuItem or null.
- Type summarize's parameter.
- Define onChoose as receiving a MenuItem.

Starter code

```
type MenuItem = { id: string; name: string; popular: boolean };
// Inside a component with useState imported:
const [title, setTitle] = useState('Menu');
const [items, setItems] = useState([]);
const [selected, setSelected] = useState(null);
// Outside the component:
const summarize = (item) => item.name;
// Define a props type for one item and an onChoose callback.
```

Plan before reading the answer

Identify the input, the output, the state owner, and the event that changes the state. Write your solution in your editor before continuing.

Hint 1: An empty array provides no useful element examples. Under common strict React TypeScript settings it can infer never[].

Hint 2: null alone does not describe a future MenuItem; use a union.

Hint 3: Return string from summarize can be inferred from the typed parameter.

Reference answer

```
// State lines go inside a component; useState must be imported.
const [title, setTitle] = useState('Menu'); // string is inferred
const [items, setItems] = useState<MenuItem[]>([]);
const [selected, setSelected] = useState<MenuItem | null>(null);

// Type declarations and this function can live outside the component.
const summarize = (item: MenuItem) => item.name;
type CardProps = {
  item: MenuItem;
  onChoose: (item: MenuItem) => void;
};
```

Inference is strongest when a value or surrounding context gives TypeScript useful evidence. A typed initial array, such as const initial: MenuItem[] = [], lets useState(initial) infer the same type. For an empty literal or null, an explicit type expresses the later states you intend.

TypeScript checks code before execution; types do not validate unknown network responses at runtime. The compiler can catch setTitle(42), but a type annotation alone cannot guarantee that downloaded JSON has the promised shape.

Test the behavior

- setItems([{ id: "a", name: "Tea", popular: true }]) type-checks; an item missing popular does not.
- setSelected(null) and setSelected(aMenuItem) are permitted.
- Before reading selected.name, use selected !== null or optional chaining; explain the possible absent case.

Transfer challenge

Define a union status type "idle" | "loading" | "ready" | "error" and explain what misspellings it catches.

E09 Build a nested layout from a verbal description

Reinforcement / 15 minutes / Flexbox, StyleSheet, box model, core components

Build a profile card. Its outer container is a column. The top row places a 64-unit avatar placeholder beside a text column. A bottom row spreads two buttons apart. Explain the main and cross axes of each row.

- Use nested containers instead of absolute positioning.
- Apply padding inside the card and margin outside it.
- Let the text column use the remaining row width.
- Use styles defined with StyleSheet.create.

Starter code

```
// Create a complete App.tsx using View, Text, Button, and StyleSheet.
// Use a colored View as the avatar so this exercise works offline.
```

Plan before reading the answer

Identify the input, the output, the state owner, and the event that changes the state. Write your solution in your editor before continuing.

Hint 1: React Native flexDirection defaults to column. Give the two horizontal containers flexDirection: "row".

Hint 2: In a row, justifyContent controls horizontal distribution and alignItems controls vertical alignment.

Hint 3: Use flex: 1 on the text column; the parent card must have available width.

Reference answer

```
// Complete App.tsx.
import { Button, StyleSheet, Text, View } from 'react-native';
export default function App() {
  return (
    <View style={styles.screen}>
      <View style={styles.card}>
        <View style={styles.top}>
          <View style={styles.avatar} />
          <View style={styles.description}>
            <Text style={styles.name}>Jordan</Text>
            <Text>First-year student learning mobile development.</Text>
          </View>
        </View>
        <View style={styles.actions}>
          <Button title="Follow" onPress={() => {}} />
          <Button title="Message" onPress={() => {}} />
        </View>
      </View>
    </View>
  );
}
const styles = StyleSheet.create({
  screen: { flex: 1, paddingTop: 48, backgroundColor: '#E8EDFF' },
  card: { margin: 16, padding: 16, borderWidth: 1,
    borderColor: '#6574A0', borderRadius: 16, backgroundColor: 'white' },
  top: { flexDirection: 'row', alignItems: 'center' },
  avatar: { width: 64, height: 64, borderRadius: 32, backgroundColor: '#815AE8'
},
  description: { flex: 1, marginLeft: 16 },
  name: { fontSize: 20, fontWeight: '700' },
  actions: { flexDirection: 'row', justifyContent: 'space-between', marginTop:
20 },
});
```

Each container controls only its direct children. The card stacks the top row and action row vertically. The top row controls the avatar and text column. The text column then stacks its own Text children. This tree makes the two-dimensional layout easier to reason about.

padding creates space between the card border and its content. margin creates space outside the card. A numeric size uses React Native logical layout units; it is not a CSS string such as "64px". The placeholder buttons intentionally have no behavior in this layout-only exercise.

Test the behavior

- Narrow the window: text wraps and remains beside the avatar.
- Change top.flexDirection to column and predict the positions before running.
- Set alignItems to flex-start and observe only the top row’s cross-axis alignment change.

Transfer challenge

Replace the placeholder with Image using a remote URI and explicit dimensions; explain why an image may need width and height.

E10 Rebuild the complete campus menu

Integration / 25 minutes / state, map, callbacks, TextInput, TypeScript, immutable arrays

Build an original campus menu app from a blank App.tsx. App owns MenuItem[]. Composer owns name and popular draft state and receives an onAdd callback. ItemCard receives one item and callbacks. Begin with two items, add new ones, toggle popularity, and remove items. Build adding first; toggling and removal are extensions if time remains. A complete reference file is provided at practice/MenuApp.tsx.

- Use one parent-owned menu array and stable IDs.
- Render cards with map and keys.
- Add a trimmed nonblank name through the child callback.
- Use functional immutable state updates.
- Show an empty state after removing every item.
- Explain each callback type aloud.

Starter code

```
type MenuDraft = { name: string; popular: boolean };
type MenuItem = MenuDraft & { id: string };
// Sketch App -> Composer and App -> ItemCard before coding.
// Start with static cards, then add state, then wire callbacks.
```

Plan before reading the answer

Identify the input, the output, the state owner, and the event that changes the state. Write your solution in your editor before continuing.

Hint 1: First render two hard-coded cards. Then replace duplicate JSX with menu.map.

Hint 2: Give Composer the type { onAdd: (draft: MenuDraft) => void }. A button calls onAdd({ name: cleanName, popular }).

Hint 3: The parent gives the draft an ID, then calls setMenu(previous => [...previous, item]). Generate the ID outside the updater so the updater remains pure.

Reference answer

```
// Complete App.tsx for a standard Expo React Native TypeScript project or
Snack.
// Original practice solution. Rebuild from the requirements before reading it.
import { useState } from 'react';
import {
  Button, Pressable, ScrollView, StyleSheet,
  Switch, Text, TextInput, View,
} from 'react-native';

type MenuDraft = { name: string; popular: boolean };
export type MenuItem = MenuDraft & { id: string };
const initialMenu: MenuItem[] = [
  { id: '1', name: 'Rice bowl', popular: true },
  { id: '2', name: 'Fruit cup', popular: false },
];

type ComposerProps = { onAdd: (draft: MenuDraft) => void };
function Composer({ onAdd }: ComposerProps) {
  const [name, setName] = useState('');
  const [popular, setPopular] = useState(false);
  const cleanName = name.trim();
  function submit() {
    if (cleanName.length === 0) return;
    onAdd({ name: cleanName, popular });
    setName('');
    setPopular(false);
  }
  return (
    <View style={styles.composer}>
      <TextInput
        accessibilityLabel="New menu item name"
        placeholder="New menu item"
        value={name}
        onChangeText={setName}
        style={styles.input}
      />
      <View style={styles.row}>
        <Text>Popular</Text>
        <Switch
          accessibilityLabel="Mark new item popular"
          value={popular}
          onValueChange={setPopular}
        />
      </View>
      <Button title="Add item" disabled={!cleanName} onPress={submit} />
    </View>
  );
}

type CardProps = {
  item: MenuItem;
  onToggle: (id: string) => void;
  onRemove: (id: string) => void;
};
```

```
function ItemCard({ item, onToggle, onRemove }: CardProps) {
  return (
    <View style={styles.card}>
      <Text style={styles.itemName}>{item.name}</Text>
      <Text>{item.popular ? 'Popular choice' : 'Regular choice'}</Text>
      <View style={styles.actions}>
        <Pressable
          accessibilityRole="button"
          accessibilityLabel={Toggle popularity of ${item.name}}
          onPress={() => onToggle(item.id)}
          style={styles.action}
        >
          <Text>Toggle popular</Text>
        </Pressable>
        <Pressable
          accessibilityRole="button"
          accessibilityLabel={Remove ${item.name}}
          onPress={() => onRemove(item.id)}
          style={styles.action}
        >
          <Text>Remove</Text>
        </Pressable>
      </View>
    </View>
  );
}

export default function App() {
  const [menu, setMenu] = useState(initialMenu);
  const [nextId, setNextId] = useState(3);
  function addMenuItem(draft: MenuDraft) {
    const item: MenuItem = { ...draft, id: String(nextId) };
    setNextId(previous => previous + 1);
    setMenu(previous => [...previous, item]);
  }
  function togglePopular(id: string) {
    setMenu(previous => previous.map(item =>
      item.id === id ? { ...item, popular: !item.popular } : item
    ));
  }
  function removeItem(id: string) {
    setMenu(previous => previous.filter(item => item.id !== id));
  }
  return (
    <View style={styles.screen}>
      <Text style={styles.title}>Campus menu</Text>
      <Text>{menu.length} items</Text>
      <Composer onAdd={addMenuItem} />
      <ScrollView contentContainerStyle={styles.list}>
        {menu.length === 0 ? (
          <Text>No items yet. Add your first one.</Text>
        ) : null}
        {menu.map(item => (
          <ItemCard
            key={item.id}
            item={item}
            onToggle={togglePopular}
            onRemove={removeItem}
          />
        ))}
```

```
      </ScrollView>
    </View>
  );
}

const styles = StyleSheet.create({
  screen: {
    flex: 1, paddingTop: 48, paddingHorizontal: 20,
    paddingBottom: 20, backgroundColor: '#F6F8FF',
  },
  title: { fontSize: 28, fontWeight: '700', marginBottom: 6 },
  composer: { paddingVertical: 16 },
  input: {
    borderWidth: 1, borderColor: '#556080', borderRadius: 8,
    padding: 12, backgroundColor: '#FFFFFF',
  },
  row: {
    flexDirection: 'row', alignItems: 'center',
    justifyContent: 'space-between', marginVertical: 8,
  },
  list: { paddingBottom: 24 },
  card: {
    padding: 16, marginBottom: 12, borderRadius: 12,
    backgroundColor: '#FFFFFF', borderWidth: 1, borderColor: '#CBD5E1',
  },
  itemName: { fontSize: 18, fontWeight: '600', marginBottom: 4 },
  actions: { flexDirection: 'row', flexWrap: 'wrap', marginTop: 8 },
  action: {
    minHeight: 44, justifyContent: 'center', paddingHorizontal: 12,
    marginRight: 8, marginBottom: 4, borderRadius: 8,
    backgroundColor: '#DDE8FF',
  },
});
```

This exercise combines rendering, state, callbacks, and TypeScript. It separates the form's temporary input from the parent's submitted data, and makes the callback contract visible in TypeScript. The parent is the single source of truth for displayed menu items.

The ID counter is state for this small in-memory practice app and each user submit creates one item. IDs are assigned once at creation rather than during rendering. A real persistent app would need an ID strategy appropriate to storage and multiple clients.

Add, toggle, and remove each return a new array. Toggle additionally creates a new object only for the changed item. The displayed count is derived from menu.length, so no second count state can drift out of sync. The screen padding is intentionally simple; production safe-area and keyboard behavior are separate concerns.

Test the behavior

- Add " Noodles ": exactly one Noodles card appears and the input clears.
- Submit a blank field: nothing is added.
- Add two items with the same name: their distinct IDs allow independent toggling and removal.
- Toggle one item twice: other items do not change.
- Remove all items: the count becomes zero and the empty message appears.
- Add an item after clearing the list: it appears normally with a new ID.

Transfer challenge

Without opening the reference, rebuild the same data flow for a reading list with title and finished fields.

E11 Fix imports and component props

Reinforcement / 8 minutes / imports, exports, components, props

You are given two local files. Fix the imports and the prop usage. Explain why default and named imports have different syntax. This drill is about modules; do not run the two files as one pasted file.

- Import the default component without braces.
- Import accent by its exported name.
- Pass the prop that CardProps declares.

Starter code

```
// Card.tsx
export type CardProps = { title: string };
export default function Card({ title }: CardProps) { /* valid JSX omitted */ }
export const accent = '#5B3FD4';

// App.tsx
import { Card } from './Card';
import accent from './Card';
// <Card name="Study" />
```

Plan before reading the answer

Identify the input, the output, the state owner, and the event that changes the state. Write your solution in your editor before continuing.

Hint 1: A default export can be given a local name by the importer.

Hint 2: A named export uses braces; aliases use as.

Hint 3: CardProps expects title, not name.

Reference answer

```
// App.tsx import and JSX fragments; Card.tsx is the separate module.
import Card, { accent } from './Card';
import type { CardProps } from './Card';
// Within JSX:
// <Card title="Study" />
// accent is available for a style, e.g. { color: accent }.

// Equally valid import with local aliases:
// import StudyCard, { accent as brandColor } from './Card';
```

A module has at most one default export and may have many named exports. The default import does not use braces. Named imports match exported names, unless an explicit alias changes the local name. import type makes it clear that CardProps is used only by TypeScript.

Props are one object parameter. Destructuring { title } extracts the title property; it does not mean the component receives an arbitrary positional string. JSX attribute names must agree with the declared props contract.

Test the behavior

- The corrected import resolves the default Card and named accent.
- TypeScript reports an error for <Card name="Study" /> because title is missing and name is not declared.
- Rename the default import to StudyCard and update the JSX; it still refers to the same component.

Transfer challenge

Split the integrated menu app into Composer.tsx, ItemCard.tsx, types.ts, and App.tsx.

E12 Debug rendering and event mistakes

Essential / 10 minutes / debugging, events, conditional rendering, keys

Repair the fragment and explain the order of events. Assume imports, count state, and the list are already declared. Do not fix a render-time update with useEffect; fix when the handler runs.

- Update only on a button press.
- Show the message only when count is greater than zero.
- Use a stable ID for each row.

Starter code

```
return (
  <View>
    <Button title="Add" onPress={setCount(count + 1)} />
    {count && <Text>There are items</Text>}
    {items.map(item => <Text key={Math.random()}>{item.name}</Text>)}
  </View>
);
```

Plan before reading the answer

Identify the input, the output, the state owner, and the event that changes the state. Write your solution in your editor before continuing.

Hint 1: A JSX prop expression runs while rendering. setCount(...) calls the setter immediately.

Hint 2: Logical && returns one of its operands; 0 && something evaluates to 0.

Hint 3: Assume each item has a stable id assigned when created.

Reference answer

```
// Component JSX fragment; count, setCount, and items already exist.
return (
  <View>
    <Button title="Add" onPress={() => setCount(c => c + 1)} />
    {count > 0 ? <Text>There are items</Text> : null}
    {items.map(item => <Text key={item.id}>{item.name}</Text>)}
  </View>
);
```

onPress expects a function that React Native can call later. Passing the result of setCount invokes the update during render and supplies no useful handler. Wrapping the call defers it until the press. A named handler passed as onPress={handleAdd} works too.

A numeric left side of && can produce the number zero rather than an element or nothing. React Native requires text to be inside Text, so prefer an explicit Boolean condition. Random keys change on every render and can remount rows, losing local state and focus.

Test the behavior

- Initial render performs no update. Pressing Add changes count once.
- At zero, no raw 0 is inserted as a View child by the condition.
- A re-render keeps the same key for each unchanged item.

Transfer challenge

Explain the difference between onPress={handleAdd}, onPress={() => handleAdd()}, and onPress={handleAdd()}.

E13 Run one interval and clean it up

Lecture breadth / 15 minutes / useEffect, timers, cleanup, hooks

Create a seconds counter that runs while running is true and pauses when false. This covers Lecture 5a. Use an effect to manage one interval and clean it up. Predict what happens after repeatedly pausing and restarting.

- Call hooks unconditionally at the top level.
- Create the interval only while running.
- Clear the interval when running changes or the component unmounts.
- Use a functional setter inside the interval.

Starter code

```
// Component-body starting point:
const [seconds, setSeconds] = useState(0);
const [running, setRunning] = useState(false);
// Add an effect and Start/Pause button.
```

Plan before reading the answer

Identify the input, the output, the state owner, and the event that changes the state. Write your solution in your editor before continuing.

Hint 1: The effect depends on running, because that value determines whether an interval exists.

Hint 2: The cleanup must return a function: return () => clearInterval(id).

Hint 3: The interval callback uses setSeconds(s => s + 1), so it does not need to capture seconds.

Reference answer

```
// Complete App.tsx.
import { useEffect, useState } from 'react';
import { Button, Text, View } from 'react-native';
export default function App() {
  const [seconds, setSeconds] = useState(0);
  const [running, setRunning] = useState(false);
  useEffect(() => {
    if (!running) return;
    const id = setInterval(() => setSeconds(s => s + 1), 1000);
    return () => clearInterval(id);
  }, [running]);
  return (
    <View style={{ padding: 40 }}>
      <Text>{seconds} seconds</Text>
      <Button title={running ? 'Pause' : 'Start'}
        onPress={() => setRunning(value => !value)} />
      <Button title="Reset" onPress={() => setSeconds(0)} />
    </View>
  );
}
```

An effect synchronizes React with a system outside rendering: here a timer. When running changes, React cleans up the previous effect before running the next setup. When the component is removed, it cleans up again. Without cleanup, repeated starts can leave multiple intervals running.

The updater receives fresh pending state, avoiding a callback that keeps incrementing an old captured seconds value. Effects with [] are mount-oriented, but development Strict Mode may perform an extra setup-cleanup-setup cycle to detect missing cleanup. Timers are not exact clocks; this is a teaching counter, not a precise stopwatch.

Test the behavior

- Start: the number increases approximately once each second.
- Pause: the number stops changing. Start again: it does not accelerate.
- Reset while running: the display becomes zero and continues on the existing interval.
- Remove the component: no interval from it remains active.

Transfer challenge

Build a countdown that stops at zero. Explain why calling setInterval in the render body would create repeated timers.

E14 Predict and chain Promises

Lecture breadth / 12 minutes / Promises, then, asynchronous ordering

Use the supplied delay function in a standalone TypeScript environment. Predict the log order, then repair the chain so the second then receives the doubled number. This models Lecture 5b without needing network access.

- Predict the synchronous log order before running.
- Return the doubled value from the first then.
- Explain that the Promise represents an eventual result.

Starter code

```
function delayValue(value: number, ms: number): Promise<number> {
  return new Promise(resolve => setTimeout(() => resolve(value), ms));
}
console.log('A');
delayValue(3, 20)
  .then(value => { value * 2; })
  .then(value => console.log('C', value));
console.log('B');
```

Plan before reading the answer

Identify the input, the output, the state owner, and the event that changes the state. Write your solution in your editor before continuing.

Hint 1: Starting an asynchronous operation does not pause the later console.log("B").

Hint 2: A then callback’s returned value becomes the fulfillment value of the next Promise in the chain.

Hint 3: Use .then(value => value * 2) or a block with return.

Reference answer

```
// Standalone TypeScript; timer scheduling affects delay, not this log order.
function delayValue(value: number, ms: number): Promise<number> {
  return new Promise(resolve => setTimeout(() => resolve(value), ms));
}
console.log('A');
delayValue(3, 20)
  .then(value => value * 2)
  .then(value => console.log('C', value))
  .catch(error => console.error('Failed:', error));
console.log('B');
// Logs A, then B, then C 6 after the timer resolves.
```

The first two logs happen in the current synchronous work. The timer later resolves the Promise with 3, allowing the then callbacks to run. then returns another Promise, so a chain represents a sequence of asynchronous and synchronous transformations.

The original block callback did not return its calculation. Its next then received undefined. This is the same missing-return idea as the broken map callback, applied to a different API. catch is an error-handling extension to the deck's basic then examples; it handles a rejection or an error thrown in the preceding chain.

Test the behavior

- The output order is A, B, C 6.
- Restore the original missing return: C receives undefined.
- Throw new Error("practice") in the first then; the catch runs and the success log is skipped.

Transfer challenge

Write a delayText function returning Promise<string>, and transform the fulfilled text to uppercase.

E15 Insert and replace with slice

Reinforcement / 10 minutes / array spread, slice, immutable arrays

This standalone TypeScript drill follows Lecture 4b slide 7. Starting with ["A", "B", "C", "D"], insert X at index 2, replace index 2 with X, and remove index 2. Each operation must start from the original array, not the previous answer.

- Leave letters unchanged.
- Explain slice's excluded end index.
- Use a new array for each result.

Starter code

```
const letters = ['A', 'B', 'C', 'D'];
const inserted = /* ? */;
const replaced = /* ? */;
const removed = /* ? */;
```

Plan before reading the answer

Identify the input, the output, the state owner, and the event that changes the state. Write your solution in your editor before continuing.

Hint 1: letters.slice(0, 2) gives A and B.

Hint 2: Insertion resumes at index 2; replacement and removal resume at index 3.

Hint 3: Spread the two slices around the inserted value.

Reference answer

```
// Standalone TypeScript.
const letters = ['A', 'B', 'C', 'D'];
const inserted = [...letters.slice(0, 2), 'X', ...letters.slice(2)];
const replaced = [...letters.slice(0, 2), 'X', ...letters.slice(3)];
const removed = [...letters.slice(0, 2), ...letters.slice(3)];
// inserted: ['A', 'B', 'X', 'C', 'D']
// replaced: ['A', 'B', 'X', 'D']
// removed: ['A', 'B', 'D']
// letters: ['A', 'B', 'C', 'D']
```

slice returns a shallow copy of the requested portion without changing the source. Its end index is exclusive. Insertion keeps the original element at index 2; replacement and removal deliberately skip it by continuing from index 3.

The different bracket roles matter: array literals use square brackets, function calls use parentheses, and object literals use braces. Naming the role of each delimiter is a faster debugging method than adding punctuation until the error disappears.

Test the behavior

- Check the three exact outputs above and confirm the original still has length 4.
- Insert at index 0 and at letters.length; predict both outputs.
- Change the index to 1 and compute the three results on paper first.

Transfer challenge

Use slice inside setTasks(previous => …) to insert a task at a chosen position.

E16 Explain a cross-platform implementation choice

Lecture breadth / 8 minutes / cross-platform, Expo, React Native, core components

This is a design-and-code interview drill. A campus club wants one small app for iOS and Android with text, images, and a sign-up form. Explain what React Native and Expo each provide, then translate the web fragment below to React Native. No networking is required.

- Use View, Text, and TextInput.
- State one shared-code benefit and one platform-specific testing need.
- Do not claim React Native is a web page inside a browser.
- Explain that Expo Go supports a fixed set of native capabilities.

Starter code

```
// Web JSX to translate:
// <div className="card"><p>Join the club</p><input /></div>
```

Plan before reading the answer

Identify the input, the output, the state owner, and the event that changes the state. Write your solution in your editor before continuing.

Hint 1: Core React Native components map to platform-native views on iOS and Android.

Hint 2: React provides the component/state model; React Native provides native UI components; Expo provides a framework, libraries, and development tools.

Hint 3: Use style objects rather than className in a standard React Native setup.

Reference answer

```
// Component JSX fragment. name and setName are useState values.
<View style={{ padding: 16, borderWidth: 1 }}>
  <Text>Join the club</Text>
  <TextInput accessibilityLabel="Your name"
    value={name} onChangeText={setName}
    style={{ borderWidth: 1, padding: 12 }} />
</View>
```

Shared TypeScript and React components can reduce duplicated application logic across iOS and Android. React Native renders platform-native views rather than requiring a WebView for the entire interface. Behavior still differs across devices, so test keyboard handling, text size, safe areas, permissions, and touch interactions on the target platforms.

Expo simplifies the React Native development workflow and offers libraries and services. Expo Go is convenient for supported APIs, but a custom native dependency can require a development build. A browser preview is useful for iteration, but it does not prove that an app behaves correctly on iOS and Android.

Test the behavior

- Explain every translated component and how its props differ from the web element.
- Identify a keyboard or layout behavior to test separately on each platform.
- Distinguish a browser web app, an installable PWA, and a React Native app in two sentences each.

Transfer challenge

Explain when a simple browser-based app might be sufficient, without assuming one approach is always better.

Mock interviews

Each simulation lasts 25 minutes. Keep solutions closed, start from a blank file, and speak your reasoning aloud. The rubric is for practice, not the course grading rubric. Confirm permitted resources for actual assessments with staff.

Menu builder rehearsal

Original 25-minute practice interview. The evaluator describes a menu screen with a title, a list of cards, and a child form that submits name and popular. These checkpoints and points are a self-study rubric, not an official course rubric. Start with the answer closed and a blank App.tsx.

- Define a typed MenuItem array and show two initial items.
- Render a custom ItemCard for each item using map and stable keys.
- Build a controlled name input and popular control in a Composer child.
- Pass an onAdd callback from the parent and append immutably.
- Reject blank input and explain the complete event-to-render path.
- If time remains, add removal or a popularity toggle.

Optional pacing

0-3: Restate the requirements, draw App with Composer and ItemCard children, and identify who owns each state value.

3-7: Define the data type and initial state; render the list and inspect the map return.

7-12: Build the controlled input and child props type; describe the callback argument.

12-19: Wire the parent callback and immutable append. Test one valid and one blank submission.

19-23: Test duplicate names and explain why IDs, not names, are keys. Attempt one extension if the core works.

23-25: Explain direct versus functional setters and identify one limitation or next test.

Interviewer hints

- First prompt: What value must the map callback return?
- Second prompt: Where is the menu state, and which function has access to its setter?
- Third prompt: A valid append updater returns [...previous, item].

Practice rubric

Data and rendering / 2 points. Uses a meaningful item type, renders data with a returning map callback, and places stable keys correctly.

State ownership and controlled input / 2 points. Keeps the menu in the parent and connects value/onChangeText correctly.

Callback contract and event timing / 2 points. Types and passes the callback, constructs the expected argument, and invokes only on an event.

Immutable update / 2 points. Appends without mutation and explains why an updater function is valid.

Reasoning and testing / 2 points. Narrates the data flow and tests blank input plus a normal add without copying a solution.

Related worked exercises: E02, E04, E07, E10

Debrief

- Record elapsed time to the first successful add and the number of hints used.
- For each mistake, state the incorrect assumption and the corrected rule.
- Close every reference and rebuild a reading-list variant later. A correct copied result is not readiness evidence.

Study task list transfer rehearsal

Original 25-minute transfer interview. Build a study-task screen with title and done fields. The task should feel structurally familiar while changing names and behavior. Attempt it after the menu rehearsal without looking at the menu code.

- Store Task[] in the parent with id, title, and done.
- Render each task through a custom row with a visible completion label.
- Let a child editor add nonblank task titles through a typed callback.
- Toggle only the selected task through an immutable map update.
- Display the number of unfinished tasks as a derived value.
- Use a nested row/column layout with deliberate padding and spacing.

Optional pacing

0-3: Sketch the component tree and list the state values versus derived values.

3-8: Create the type, seed data, and task rows. Explain stable identity.

8-14: Wire the child editor and parent add handler with a functional update.

14-19: Implement toggle by ID, copying the matching object.

19-23: Derive the unfinished count and test two independent tasks.

23-25: Explain why a shallow array copy alone would not protect an edited task object.

Interviewer hints

- First prompt: What should remain unchanged when only one task changes?
- Second prompt: A row can receive an onToggle callback and call it with its ID.
- Third prompt: Use previous.map with a conditional object spread.

Practice rubric

Independent transfer / 2 points. Builds a task-oriented solution without copying or merely renaming the menu reference.

State and derived values / 2 points. Uses one tasks state and derives the unfinished count from it.

Typed callback flow / 2 points. Child requests add/toggle through explicit compatible callback props.

Immutable object edit / 2 points. Copies the matching object and preserves other tasks.

Layout and debugging / 2 points. Uses nested containers and tests toggling twice plus blank input.

Related worked exercises: E05, E07, E09, E10

Debrief

- Compare the data flow with the menu app and explain what transferred.
- Identify any step completed only by recalling punctuation rather than understanding its role.
- Repeat a smaller version the next day, then add removal without a hint.

State and asynchronous debugging rehearsal

Original 25-minute breadth interview. Repair small state, timer, and Promise examples and explain their behavior before running them. This practice covers the later supplied lectures and complements the component-building rehearsals.

- Predict two direct increments versus two updater increments.
- Repair a render-time handler call and a missing map return.
- Build a start/pause timer with one effect and cleanup.
- Predict Promise log order and repair a missing then return.
- Explain hook placement and why [] is not a guarantee of one setup in development.

Optional pacing

0-5: Predict the queued state examples and explain your reasoning aloud.

5-9: Repair the missing-return map from e02 and the JSX/event fragment from e12. Describe what triggers each bug.

9-18: Implement the interval effect from requirements, then test pause/restart behavior.

18-22: Predict and repair the Promise chain in e14.

22-25: Explain cleanup, dependencies, and one useful debugging experiment.

Interviewer hints

- First prompt: Which values belong to the current render snapshot?
- Second prompt: What resource must be stopped when this effect is replaced?
- Third prompt: A block body needs return when it should provide a value.

Practice rubric

State prediction / 2 points. Correctly explains snapshot values and queued updater composition.

Event and map debugging / 2 points. Fixes immediate invocation and missing return without unrelated changes.

Timer lifecycle / 2 points. Creates one interval while running and returns cleanup.

Promise reasoning / 2 points. Predicts ordering and passes the returned value through the chain.

Explanation and validation / 2 points. Connects each fix to an observable test and acknowledges dev effect behavior.

Related worked exercises: E03, E02, E12, E13, E14

Debrief

- List each prediction that differed from execution and explain why.
- Distinguish a stale closure bug from a leaked interval; they need different fixes.
- If the core menu rehearsal is still weak, return to e01-e08 before adding asynchronous complexity.

Interview question bank

Q01 Arrow functions

What do n => n * 2 and n => { n * 2; } return for n = 5?

Hint: Name the two forms of function body.

The expression body returns 10. The block body computes 10 but returns undefined because it has no return statement. Adding return inside the block makes the functions equivalent for this input. Parentheses around a multiline expression improve grouping but do not replace a required return in a block.

Follow-up: Rewrite both valid forms with the parameter type number.

Q02 Objects

Why does () => ({ popular: true }) return an object?

Hint: The parentheses distinguish an expression from a statement block.

After =>, a bare opening brace starts the function body. Wrapping { popular: true } in parentheses makes it an object expression, which the arrow returns implicitly. An equivalent block form is () => { return { popular: true }; }.

Follow-up: Write a function that receives name and returns { name, popular: false }.

Q03 map

Given [2, 5, 8].map(n => n + 1), what is returned and what happens to the original?

Hint: Follow one input element at a time.

The result is a new array [3, 6, 9]. The original remains [2, 5, 8] because this callback only computes values. map itself does not require mutation, but a callback can still mutate an object if written carelessly. For a normal dense array, each input element contributes one returned value.

Follow-up: What does map produce if its callback returns nothing?

Q04 JSX

What is the difference between the braces in {items.map(…)} and the braces in item => { return …; }?

Hint: The same character has different jobs in different positions.

The JSX braces enter a JavaScript expression within markup. The arrow braces create a function statement block, where return is needed to supply a value. An object literal also uses braces. Debugging starts by identifying each brace's role rather than treating all braces as interchangeable.

Follow-up: Name the roles of every delimiter in setItems(prev => [...prev, item]).

Q05 TypeScript inference

Why can useState(initialItems) infer Item[], while useState([]) may need an explicit type?

Hint: Ask what information is available in each initial value.

If initialItems is declared Item[], useState receives that evidence and infers the state type. An empty literal gives no element examples; in common strict React TypeScript settings useState([]) can infer never[]. useState<Item[]>([]) states the intended element type. Inference is useful, but it cannot guess all future values.

Follow-up: What annotation would you use for a selected item that initially has no value?

Q06 TypeScript boundaries

Does type User = { name: string } guarantee a downloaded JSON object has a name?

Hint: Separate static checking from runtime execution.

No. TypeScript's type annotations are removed before execution and do not inspect downloaded values. A type assertion merely tells the compiler what to assume. Unknown external data needs runtime checks or validation before the app trusts its shape. This is a conceptual extension beyond the supplied Promise examples.

Follow-up: What simple check would you perform before reading an unknown value as a name?

Q07 State

Why is let count = 0 inside a component insufficient for an interactive counter?

Hint: Consider both updating the screen and remembering a value.

Changing an ordinary local variable does not ask React to render again. Also, the component function runs again on a later render and recreates the local initialization. useState retains a value across renders and provides a setter that schedules an update. A module-level variable survives calls but still does not notify React, and may be unintentionally shared.

Follow-up: What happens to a local variable if a parent causes the component to render again?

Q08 State snapshots

Starting from count = 0, a handler calls setCount(count + 1) twice. What is the next count?

Hint: Both expressions read the same render's count.

The next count is 1. Each call asks for the value 1 computed from the captured zero. Two calls to setCount(c => c + 1) instead compose to 2 because each updater receives the pending result of the previous update. The current handler still reads its original snapshot after scheduling updates.

Follow-up: Predict setCount(5); setCount(c => c + 1) from an initial count of zero.

Q09 Setter forms

Is setMenu(previous => [...previous, item]) valid? What is wrong with setMenu(previous => { ...previous, item })?

Hint: Keep the updater-function idea and inspect the returned value.

It is valid and is useful when the next array depends on the previous array. A React setter accepts a next value or a pure updater function. The attempted code used braces and spread in an invalid block rather than returning a new array. Square brackets form the array; an expression body returns it automatically.

Follow-up: Show the equivalent block-body updater and the direct replacement form.

Q10 Immutable updates

Is const next = [...items]; next[0].popular = true an immutable update?

Hint: What exactly was copied?

The array was copied, but its objects are still shared. Changing next[0].popular mutates the same object referenced by items[0]. For a change to one item, use map and return { ...item, popular: true } for the matching item. Copy each level whose contents change. Unchanged objects may retain their references.

Follow-up: Why is [...items, newItem] sufficient when you are only appending?

Q11 Callbacks

Explain onPress={save}, onPress={() => save()}, and onPress={save()}.

Hint: Distinguish a function value from the result of a call.

The first passes save itself. The second passes a new function that calls save when pressed. The third calls save during rendering and supplies its return value as the prop, which is usually wrong. A wrapper is especially useful when passing a chosen argument, such as () => removeItem(item.id).

Follow-up: When can onChangeText={setName} be used directly?

Q12 Lifting state

Two sibling components must display and change the same selected item. Where should the state live?

Hint: Locate their nearest common ancestor.

Store it in their nearest common ancestor, pass the current selection down as props, and pass callbacks that request changes. This gives one source of truth. Keeping independent copies in each sibling creates a synchronization problem. State can remain local when no other component needs it.

Follow-up: Why might a form’s unsent draft stay in the form while submitted items live in the parent?

Q13 Function props

Explain the type onAdd: (item: MenuItem) => void.

Hint: Read it as a contract for a value that is a function.

onAdd is a property containing a function. The function accepts one MenuItem argument. Its caller does not use a returned result, represented by void. The parent supplies a compatible function; the child can call it with an object that includes all required MenuItem fields. The type declaration does not run the function.

Follow-up: How would the type change if the callback accepted only an item ID?

Q14 Controlled inputs

The TextInput has value={name}, but typing does not update the display. What do you inspect?

Hint: Trace the input-to-state-to-prop loop.

Check onChangeText and confirm it receives the new string and calls the setter for name. Verify the input's value is not hard-coded or derived from another unchanged state. Also inspect whether an effect or parent update resets it. A controlled input displays its supplied value, so user edits must update the state that supplies that value.

Follow-up: Why is event.target.value usually the wrong expression for onChangeText?

Q15 Keys

Why can an array index be a poor key for an editable or reorderable list?

Hint: A key identifies an item across renders, not merely its current position.

If items are inserted, deleted, or reordered, the same index can refer to a different item. React may associate row state or focus with the wrong data. Use a stable unique ID stored with each item. Random keys are also poor because they force new identities every render. A key is not automatically passed as a normal child prop.

Follow-up: Where should key go when map returns a custom ItemCard?

Q16 Flexbox

In a row container, what do justifyContent and alignItems control? What changes in a column?

Hint: Start by drawing the main axis.

justifyContent distributes children along the main axis. In a row that axis is horizontal; in a column it is vertical. alignItems aligns children across the cross axis, which is perpendicular. React Native defaults to column, unlike the familiar web flex-direction default of row. The parent's size must allow visible free space for distribution to be obvious.

Follow-up: Which container should receive these styles in a nested card layout?

Q17 Box model

What is the difference between padding, margin, and borderWidth?

Hint: Trace from content outward.

Padding creates space inside the border between the content and the edge. The border surrounds that padded content. Margin creates space outside the border between neighboring layout boxes. Use padding for a comfortable touchable interior and margin for spacing between cards. React Native style property names use camelCase and most numeric dimensions are logical units.

Follow-up: Why does margin not increase the interior of a card’s background?

Q18 Core components

When would you use View, Text, Image, TextInput, and ScrollView?

Hint: Match each component to one job.

View groups and lays out content. Text displays text and supports nested styled text. Image displays an image from a source and often needs explicit dimensions. TextInput gathers editable text, usually using state. ScrollView makes bounded content scrollable and renders its children together. For very large lists, FlatList is a useful extension because it renders a window of rows.

Follow-up: Why should arbitrary text not be placed directly inside View?

Q19 Modules

What is wrong with import { Card } from "./Card" if Card.tsx exports default function Card()?

Hint: Braces indicate a named import.

The module provides a default export, so use import Card from "./Card". The local name could also be MenuCard. Named exports use braces and match the exported name unless explicitly aliased. Type-only exports can be imported with import type. Package imports resolve installed dependencies; relative imports resolve local files.

Follow-up: Show a single statement importing a default Card and a named color.

Q20 Hooks

Why should useState not be called inside an if statement?

Hint: React depends on consistent hook ordering between renders.

If the condition changes, the component calls a different sequence of hooks. React can no longer associate each hook position with the same state or effect. Call hooks at the top level of a React function component or custom hook, before conditional returns. Place conditional behavior inside an effect or event handler instead of around the hook call.

Follow-up: How would you run an effect only while running is true without conditionally calling useEffect?

Q21 Effects

Compare no dependency array, [], and [running] for useEffect.

Hint: Each case still involves setup after a committed render.

Without a dependency array the effect runs after each commit of that component. With [], it is tied to mounting rather than changed reactive dependencies. With [running], it runs on mount and after commits where running changes. Cleanup runs before a replacement setup and on unmount. Development Strict Mode may add an extra setup-cleanup cycle, so [] is not a universal promise of exactly one call.

Follow-up: Why should every reactive value used by an effect be considered for its dependencies?

Q22 Timer cleanup

Why does a counter accelerate after the user repeatedly opens and closes a screen?

Hint: Count how many intervals may still be alive.

A likely cause is an interval created without cleanup, so old timers survive while new timers are added. Keep the interval ID inside an effect and return () => clearInterval(id). Also inspect whether a render body starts an interval on every render. A functional state updater avoids a separate stale-count problem, but it does not replace cleanup.

Follow-up: What does setSeconds(s => s + 1) solve, and what does clearInterval solve?

Q23 Promises

What is logged by console.log("A"); Promise.resolve(2).then(n => console.log(n)); console.log("B")?

Hint: A fulfilled Promise’s then callback is still scheduled asynchronously.

The order is A, B, 2. The synchronous statements finish before the Promise reaction runs. A Promise represents eventual fulfillment or rejection; then attaches a reaction and returns another Promise. Returning a value from a then callback passes it to the next stage; returning another Promise lets the chain wait for that Promise’s result.

Follow-up: What happens if a then callback uses braces but forgets return?

Q24 Cross-platform development

What can be shared in a React Native app, and what must still be tested per platform?

Hint: Separate shared source code from runtime behavior.

Components, state logic, TypeScript types, and much styling can be shared. Native controls, keyboard behavior, safe areas, permissions, text scaling, and library support can differ. Expo provides tooling and libraries around React Native; it does not remove the need for device testing. React Native is not simply a browser page packaged as a mobile app.

Follow-up: When might a custom native dependency require a development build rather than Expo Go?

Readiness evidence

- Without hints, write a typed arrow function in expression and block forms and explain the return value of each.
- From a blank fragment, render a typed data array through a custom component with a correct map return and stable keys.
- Explain and write both valid setter forms. Append twice through functional updates without losing either item.
- Toggle one object without mutating the old array or old object, and remove by ID with a correct predicate.
- Build a controlled TextInput and parent-owned list connected through a typed callback without looking at the integrated solution.
- Restate a callback's input object shape before constructing it, and explain why the handler runs on a press rather than during rendering.
- Use inference for useful initial values and explicit types for empty arrays, nullable state, and standalone function parameters.
- Complete the core of a 25-minute original mock, record any hints, and repeat the key skill later without those hints. A practice score tracks learning; it is not an official course result.
- Pass a transfer attempt using new fields and requirements, such as tasks with completion instead of menu items with popularity.
- Explain nested Flexbox axes, core component choices, box-model spacing, and default versus named imports.
- Predict state batching, effect cleanup, and Promise log order, then confirm the prediction with a focused experiment.
- Use the answer key and code cheat sheet after attempting the problem. Close them before a timed reconstruction or transfer task.

Read more: React useState / React Rendering Lists / React Updating Arrays in State / React Sharing State Between Components / React useEffect / TypeScript Everyday Types / React Native Layout with Flexbox